

# myoArm Forward-State-Estimation 研究遂行テキスト

理系大学院生が Project 1 を理解し、再現し、次フェーズへ拡張するために

作成日: 2026-05-12 / 対象: 修士・博士前期～博士後期初期 / Repository: myoarm-forward-state-estimation

## このテキストの目的

この資料は、myoArm forward-state-estimation 研究を「読む」だけでなく、実際に再現し、結果を検証し、次の Project 2 / Project 3 に発展させるための作業教科書である。読者は Python / PyTorch / 基礎的な制御・確率の知識を持つ理系大学院生を想定する。

外部画像は使わず、リポジトリ内の生成図表と既存 primer 画像のみを用いる。

## 目次

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| 1. 研究の全体像 .....                                                                  | 2  |
| 1.1. 研究群の中での位置づけ .....                                                           | 2  |
| 1.2. 中心問い .....                                                                  | 3  |
| 1.3. この資料の読み方 .....                                                              | 3  |
| 1.4. なぜ closed-loop pipeline を考えるのか .....                                        | 4  |
| 1.5. このループは妥当なのか .....                                                           | 4  |
| 1.6. Downstream policy の 3 分類 .....                                              | 5  |
| 1.7. Closed-loop pipeline を時刻順に読む .....                                          | 5  |
| 1.8. 1 step の擬似コード .....                                                         | 6  |
| 1.9. delay がない場合とある場合 .....                                                      | 7  |
| 1.10. correction gain $K$ の直感 .....                                              | 7  |
| 1.11. logger に全層を保存する理由 .....                                                    | 7  |
| 2. 必要な前提知識 .....                                                                 | 8  |
| 2.1. 筋骨格 reaching と MyoSuite .....                                               | 8  |
| 2.2. 信号名の区別 .....                                                                | 9  |
| 2.3. 状態ベクトル .....                                                                | 9  |
| 3. Forward model .....                                                           | 9  |
| 3.1. なぜ forward model が必要か .....                                                 | 9  |
| 3.2. 残差 MLP .....                                                                | 10 |
| 3.3. one-step と multi-step supervision .....                                     | 10 |
| 4. Predictive state observer .....                                               | 10 |
| 4.1. 基本式 .....                                                                   | 10 |
| 4.2. fixed-lag delay handling .....                                              | 11 |
| 4.3. 二層 reliability-adaptive observer .....                                      | 11 |
| 4.3.1. Within-trial layer: innovation history から reliability へ .....             | 12 |
| 4.3.2. Across-trial layer: SPSA outcome adaptation .....                         | 12 |
| 4.3.3. oracle-supervised predictor: upper bound diagnostic (Appendix C 教材) ..... | 13 |
| 5. 実験 pipeline .....                                                             | 14 |
| 5.1. Repository の構造 .....                                                        | 14 |
| 5.2. 最小再現手順 .....                                                                | 14 |
| 5.3. 実験 command の読み方 .....                                                       | 14 |
| 6. 主要結果の読み方 .....                                                                | 14 |
| 6.1. C1: default reliability is task-misaligned .....                            | 15 |
| 6.2. C2: single-cell SPSA closes most of the gap .....                           | 16 |

|       |                                                            |    |
|-------|------------------------------------------------------------|----|
| 6.3.  | C3: multi-cell SPSA reveals parameterisation ceiling ..... | 16 |
| 6.4.  | C4: task transfer reveals task-dependent rule .....        | 18 |
| 7.    | 研究者としての作業手順 .....                                          | 18 |
| 7.1.  | 1日の作業サイクル .....                                            | 18 |
| 7.2.  | 実験前チェックリスト .....                                           | 18 |
| 7.3.  | 結果解釈チェックリスト .....                                          | 19 |
| 8.    | よくある失敗と対処 .....                                            | 19 |
| 8.1.  | target が paired でない .....                                  | 19 |
| 8.2.  | K と H の混同 .....                                            | 19 |
| 8.3.  | behaviour-cloned policy の open-loop 化(Appendix C 教材) ..... | 19 |
| 8.4.  | endpoint-error policy を optimal controller と呼ぶ .....       | 19 |
| 8.5.  | SPSA outcome を training-time と deployed eval で混同する .....   | 20 |
| 8.6.  | oracle なしを “no oracle access” と書く .....                    | 20 |
| 9.    | Project 2 / Project 3 への接続 .....                           | 20 |
| 9.1.  | Project 1.5: context-conditioned $\beta$ network .....     | 20 |
| 9.2.  | Project 2: Bayesian integration .....                      | 20 |
| 9.3.  | Project 3: cortico-cerebellar loop .....                   | 21 |
| 10.   | 演習 .....                                                   | 21 |
| 10.1. | 演習 1: 状態 schema を確認する .....                                | 21 |
| 10.2. | 演習 2: test を通す .....                                       | 21 |
| 10.3. | 演習 3: 図表を読み解く .....                                        | 21 |
| 10.4. | 演習 4: 新しい transfer test を設計する .....                        | 22 |
| 10.5. | 演習 5: context-conditioned $\beta$ の prototype を設計する .....  | 22 |
| 11.   | 参考資料 .....                                                 | 22 |
| 11.1. | Repository 内 .....                                         | 22 |
| 11.2. | Obsidian 内 .....                                           | 22 |
| 11.3. | 文献 .....                                                   | 22 |
| 12.   | 最後に .....                                                  | 23 |

## 1. 研究の全体像

### 1.1. 研究群の中での位置づけ

本研究は myoArm 新規研究群の Project 1 である。旧 myoarm-lambda-ep の lambda-EP 単独路線から離れ、重力下の筋骨格 reaching における forward prediction、遅延感覚 feedback、不確実性統合を検証する流れの最初の本体プロジェクトである。

| Project   | 中心問い                                      | Project 1 との関係                                          |
|-----------|-------------------------------------------|---------------------------------------------------------|
| Project 0 | 共通 myoArm 基盤                              | target set、logger、noise/delay wrapper、metrics を提供       |
| Project 1 | forward model + predictive state observer | 本資料の対象                                                  |
| Project 2 | Bayesian reliability-weighted integration | 視覚・固有受容・prediction・prior の信頼度統合へ拡張                      |
| Project 3 | cortico-cerebellar loop                   | residual MLP forward model を cerebellar-like module に置換 |
| Project 4 | cortical population dynamics              | M1-like population dynamics へ拡張                         |

#### 研究ノート

Project 1 は「完成した単独テーマ」であると同時に、後続 Project の土台である。したがって、実装では再利用可能な状態表現・logger・評価 pipeline を作ることが重要になる。

## 1.2. 中心問い

Project 1 の中心問いは次である。

### 中心問い

myoArm reaching において、forward-model-based predictive state observer は、agent 自身が online で生成できるシグナル(innovation history と per-episode reaching outcome)だけから、correction gain を self-adapt できるか。

神経科学的には、これは小脳 forward model、efference copy、遅延感覚 feedback、sensory prediction error の追跡、感覚信頼度の online 推定、そして reaching outcome に基づく試行間学習を、筋骨格腕シミュレーション上で検証する第一段階である。生体は per-condition の swept oracle( $K^*$  ラベル)を持たないので、agent-available signal だけで correction gain が決まりうるかは、forward-model framework の生物学的妥当性を問う中心的な実験となる。

工学的には、これは以下の問題になる。

前の推定状態  $\hat{x}_t$  と、前に出した運動指令  $u_t$

-> forward model

-> 次時刻の予測状態  $\hat{x}_{t+1}$

遅延・ノイズ付き観測  $y_{t+1}$

-> sensory prediction-error correction

-> 更新後の推定状態  $\hat{x}_{t+1}$

$\hat{x}_{t+1}$

-> controller

-> 次の運動指令  $u_{t+1}$

-> muscle excitation

-> myoArm plant

つまり、同じ  $\hat{x}$  でも「更新前」と「更新後」がある。混乱を避けるため、本資料では 1 step の流れを  $\hat{x}_t \rightarrow \hat{x}_{t+1} \rightarrow u_{t+1}$  と読む。

ここで  $\hat{x}_{t+1}$  と  $\hat{x}_t$  は役割が違う。

| 記号              | 意味                                       | どう作るか                                                                                |
|-----------------|------------------------------------------|--------------------------------------------------------------------------------------|
| $\hat{x}_{t+1}$ | forward model だけで作った「次はこうなっているはず」という予測状態 | $\hat{x}_t$ と $u_t$ を forward model に入れる                                             |
| $\hat{x}_{t+1}$ | 観測も使って補正した「controller に渡す最終的な推定状態」       | $\hat{x}_{t+1}$ と delayed/noisy observation $y_{t+1}$ の差分を correction gain $K$ で補正する |

### 直感

$\hat{x}_{t+1}$  は「自分のモデルだけでの予想」。

$\hat{x}_t$  は「予想に、届いた感覚情報を加味して更新した現在の信念」。

controller が使うのは  $\hat{x}_{t+1}$  ではなく、基本的には更新後の  $\hat{x}_t$  である。

## 1.3. この資料の読み方

この資料は、最初から C1-C4 の論文主張を読む構成にはしていない。まず closed-loop pipeline、状態表現、forward model、predictive state observer、そして二層適応則(within-trial reliability と across-trial outcome adaptation)の意味を理解し、その後で結果 C1-C4 を読む。

推奨順序:

1. closed-loop pipeline が何のためにあるか
2.  $\hat{x}_t$  /  $\hat{x}_{t+1}$  / observation / action の違い

3. forward model が何を予測するか
4. sensory prediction-error correction と gain  $K$  の意味
5. innovation history からの within-trial reliability
6. SPSA across-trial outcome adaptation
7. oracle-supervised upper bound (Appendix 教材)
8. C1-C4 の結果

oracle-supervised predictor(旧 §3.4)は upper-bound diagnostic として Appendix C 化された。Project 1 の中心提案は、oracle ラベルを使わない二層適応 observer である。

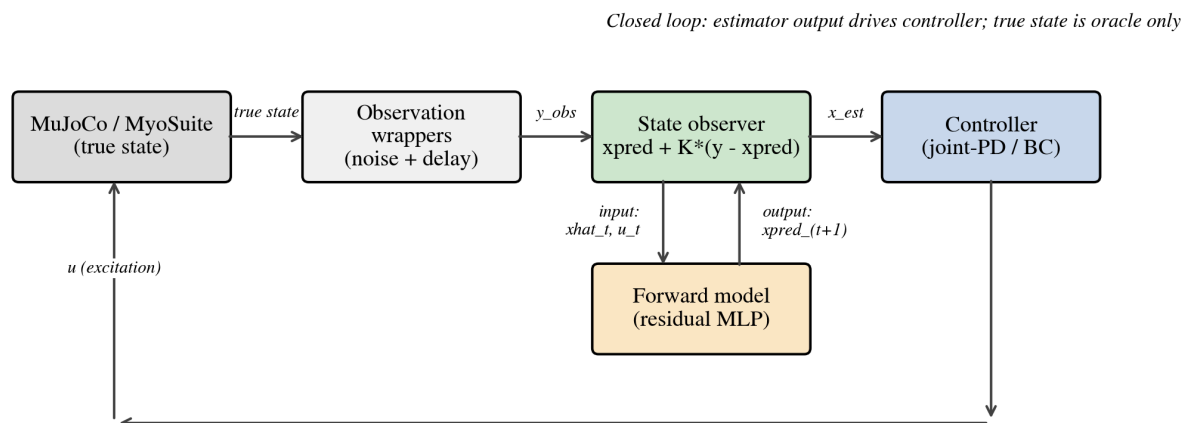


図 1: Project 1 の closed-loop pipeline。true state は評価専用で、controller は estimator output のみを見る。

#### 1.4. なぜ closed-loop pipeline を考えるのか

この pipeline は、単にソフトウェアを複雑にしたいから作っているのではない。研究上の目的は、状態推定の質が、実際の筋骨格 reaching の制御性能に伝播するかを調べることである。

open-loop evaluation では、記録済み trajectory を replay しながら  $\hat{x}_t$  と  $x_t$  の誤差を測る。これは forward model や estimator の純粋な精度を見るには有用だが、次の問いには答えられない。

##### closed-loop でしか答えられない問い

推定状態  $\hat{x}_t$  が少し良くなったとき、controller が実際に違う muscle excitation を出し、腕の軌道や到達誤差が変わるのか。

この問いに答えるには、estimator を controller の前段に置き、推定誤差が action を通じて plant に戻る循環を作る必要がある。これが closed-loop pipeline である。

| 評価          | 測れること                                              | 測れないこと                      |
|-------------|----------------------------------------------------|-----------------------------|
| open-loop   | state estimation error, prediction MSE, oracle $K$ | その推定が行動に効くか                 |
| closed-loop | reaching error, success, overshoot, 推定誤差の行動への伝播    | 純粋な estimator 精度だけの切り分けは難しい |

したがって、Project 1 では open-loop と closed-loop を両方使う。open-loop は estimator の診断、closed-loop は「その推定が制御に意味を持つか」の検証である。

#### 1.5. このループは妥当なのか

この closed-loop は、神経科学的にも制御工学的にも妥当な抽象化である。ただし「脳を完全再現している」という意味ではない。

| 観点 | 妥当な理由 | 限界 |
|----|-------|----|
|----|-------|----|

|      |                                                                                    |                                                          |
|------|------------------------------------------------------------------------------------|----------------------------------------------------------|
| 神経科学 | efference copy に基づく forward prediction と delayed sensory feedback の統合を表す           | 小脳 microcircuit, spike, climbing fiber learning は再現していない |
| 制御工学 | observer + controller という標準的な閉ループ構造                                                | gain は scalar で、完全な covariance 推定から導出した最適 gain ではない      |
| 実験設計 | true state を評価専用にし、controller は estimated state だけを見るため、state estimation の効果を検証できる | controller の質が低いと estimator 差が task に出ない                 |

Project 1 の closed-loop は、以下の仮説を検証するための最小構成である。

delayed/noisy sensory feedback  
+ forward prediction  
+ sensory prediction-error correction  
-> better estimated current state  
-> different controller action  
-> different reaching trajectory

この連鎖のどこかが切れると、状態推定が良くても task performance には出ない。実際、behaviour-cloned policy では reaching は改善したが state coupling が弱く、observer signal は wash out した。逆に joint-space feedback controller と endpoint-error sensorimotor policy は state-coupled なので、推定状態の差が task 指標に現れた。

#### 研究ノート

closed-loop pipeline の妥当性は「controller が良い」ことではなく、「controller が推定状態に依存している」ことにある。Project 1 の主結果は、状態推定の効果が state-coupled controller と observation-delay regime の組み合わせで現れる、という点である。

## 1.6. Downstream policy の 3 分類

論文が一段落した時点で、controller 軸は次の 3 分類に整理された。ここでは controller を広い意味で使うが、BC と endpoint-error は policy と呼ぶ方が正確である。

| 名前                                 | 何を見るか                                                  | 位置づけ                                             |
|------------------------------------|--------------------------------------------------------|--------------------------------------------------|
| joint-space feedback controller    | joint error / velocity error から muscle excitation を出す  | hand-designed, joint-level, state-coupled        |
| endpoint-error sensorimotor policy | estimated tip-to-target error から muscle excitation を出す | task-level, endpoint-error-driven, state-coupled |
| behaviour-cloned policy            | demonstration から state-to-action mapping を学習する         | minimal learned sensorimotor policy              |

endpoint-error sensorimotor policy は、OFC-like controller や LQR-like controller ではない。Riccati equation、LQR、OFC を解いているわけではなく、推定された手先誤差に基づいて筋入力を補正する transparent な task-level policy である。

#### 研究ノート

この 3 分類の意味は、「どの policy が一番 reaching できるか」だけではない。observer の出力  $\hat{x}_t$  が downstream policy の action に実際に伝わるか、つまり state coupling があるかを見るための軸である。

## 1.7. Closed-loop pipeline を時刻順に読む

Fig. 1 の closed-loop pipeline は、1 control step ごとに同じ処理を繰り返す。重要なのは、シミュレータ内部には常に真の状態  $x_t$  があるが、controller はそれを直接見ないという点である。controller が見るのは、delay/noise wrapper と estimator を通った  $\hat{x}_{t|t}$  だけである。

| 順序 | モジュール | 入力 -> 出力 | 研究上の意味 |
|----|-------|----------|--------|
|----|-------|----------|--------|

|   |                                       |                                                                                            |                                                                   |
|---|---------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| 1 | MuJoCo / MyoSuite plant               | $u_{(t-1)} \rightarrow \text{true state } x_t$                                             | 身体が実際にどう動いたか。これは評価用 oracle であり controller には渡さない。                 |
| 2 | state extractor                       | $\text{mj\_data} \rightarrow \text{MyoArmState}(x_t)$                                      | qpos, qvel, act, tip_pos, target_pos, reach_err を研究用 schema に整える。 |
| 3 | observation wrappers                  | $x_t \rightarrow \text{delayed/noisy observation } y_t$                                    | 感覚遅延・観測ノイズを明示的に入れる。生体の sensory feedback 制約に対応。                    |
| 4 | forward model                         | $\hat{x}_{(t-1)}, u_{(t-1)} \rightarrow \text{prediction } x_{\text{pred}_t}$              | efference copy と前状態推定から現在状態を予測する。                                 |
| 5 | predictive state observer             | $x_{\text{pred}_t}, y_t, \text{correction gain } K \rightarrow \text{estimate } \hat{x}_t$ | sensory prediction error を補正し、controller が使う現在状態を作る。              |
| 6 | controller                            | $\hat{x}_t \rightarrow \text{neural command / excitation } u_t$                            | 推定状態に基づいて次の筋入力を決める。                                               |
| 7 | ActionAdapter + SDN                   | $\text{excitation} \rightarrow \text{api\_action}$                                         | 必要なら signal-dependent motor noise を入れ、Gym API 形式へ変換する。            |
| 8 | $\text{env.step}(\text{api\_action})$ | $\text{api\_action} \rightarrow \text{next plant state}$                                   | 次の時刻へ進む。                                                          |
| 9 | logger / metrics                      | true_state, observation, estimate, action                                                  | 評価・学習・再現のために全層を保存する。                                              |

### 確認ポイント

closed-loop の核心は、true\_state  $\rightarrow$  observation  $\rightarrow$  estimator  $\rightarrow$  controller  $\rightarrow$  action  $\rightarrow$  plant の循環である。true\_state は metrics と supervised training には使うが、closed-loop controller の入力にはしない。

## 1.8. 1 step の擬似コード

実装の概念は次のように読める。これは実際の関数名を完全に写したものではなく、責務を理解するための擬似コードである。

```
# t 時点。env 内部には真の MuJoCo state がある。
true_state = extract_state(env)      # oracle for logging/evaluation only

# 生体という sensory feedback。ここで delay/noise が入る。
obs_state = delayed_wrapper.observe(true_state)
obs_state = noisy_wrapper.observe(obs_state)

# 前 step の推定状態と実際に入れた muscle excitation から現在を予測。
x_pred = forward_model.predict(prev_estimate, prev_excitation)

# sensory prediction error を correction gain K で補正。
estimate = estimator.update(prediction=x_pred, observation=obs_state, gain=K)

# controller は estimate だけを見る。true_state は見ない。
neural_command = controller(estimate)
excitation = command_to_excitation(neural_command)
excitation = motor_noise(excitation)  # optional SDN
api_action = action_adapter.to_api(excitation)

# plant を 1 step 進める。
obs, reward, terminated, truncated, info = env.step(api_action)

# 後で検証できるように、true / obs / estimate / action を全部保存。
logger.append(
    true_state=true_state,
    obs_state=obs_state,
    estimate=estimate,
```



```

neural_command=neural_command,
excitation=excitation,
api_action=api_action,
)

```

このコードで最も大切なのは、`controller(estimate)` であって `controller(true_state)` ではないことだ。true state controller は oracle baseline としては有用だが、Project 1 の本筋ではない。

## 1.9. delay がない場合とある場合

delay が  $d=0$  の場合、observation  $y_t$  は現在状態  $x_t$  の noisy version と見なせる。このとき prediction-error correction は直感的である。

```

x_pred_t = forward_model(xhat_(t-1), u_(t-1))
y_t      = x_t + noise
xhat_t   = x_pred_t + K * (y_t - x_pred_t)

```

delay が  $d>0$  の場合、 $y_t$  は現在ではなく過去  $x_{(t-d)}$  の観測である。したがって、単純に  $x_{pred\_t}$  と  $y_t$  を混ぜると時刻がずれる。Project 1 では fixed-lag buffer を使い、過去の estimate を correction してから現在まで re-roll する。

1. buffer 内に  $xhat_{(t-d)}$ , ...,  $xhat_{(t-1)}$  と action を保持
2.  $y_t$  は  $x_{(t-d)}$  の観測なので、 $xhat_{(t-d)}$  に対して correction
3. corrected  $xhat_{(t-d)}$  を  $u_{(t-d)}$ , ...,  $u_{(t-1)}$  で forward model rollout
4. 現在時刻の estimate  $xhat_t$  を得る

### 落とし穴

delay があるときに  $y_t$  を現在状態として扱うと、推定器は「古い身体状態」を現在だと思って制御する。この誤りは reaching の振動・遅れ・overshoot として現れる。

## 1.10. correction gain $\kappa$ の直感

ここでの  $\kappa$  は Kalman filter から導出した gain ではない。 $\kappa$  は sensory prediction error に対する correction gain であり、forward prediction と sensory feedback のどちらをどの程度信じるかを表す実験上の knob である。

| K            | 推定器の挙動                                                      | closed-loop で起きやすいこと                                    |
|--------------|-------------------------------------------------------------|---------------------------------------------------------|
| K=0          | prediction-only。observation correction を使わない。               | forward model が強ければ delay を回避できるが、model error があると発散する。 |
| K=1          | observation-only。prediction は correction 後の re-roll 以外では弱い。 | 観測が新鮮なら強い。delay が大きいと古い情報に引っ張られる。                       |
| $0<\kappa<1$ | prediction と observation の blending。                        | 観測ノイズが大きく、delay が小さい条件では有利になりやすい。                       |

この研究で重要だったのは、open-loop state estimation error を最小にする  $\kappa$  と、closed-loop task error を最小にする  $\kappa$  が一致しない場合があることだった。つまり「状態推定として正確」でも「制御に使うと良い」とは限らない。

## 1.11. logger に全層を保存する理由

closed-loop pipeline では、1 step ごとに似たような vector が複数出てくる。これらを保存しないと、失敗したときに原因を切り分けられない。

| 保存する量          | 用途                                      | 典型的な診断                     |
|----------------|-----------------------------------------|----------------------------|
| true_*         | oracle evaluation / supervised training | estimation error が本当に下がったか |
| obs_*          | delay/noise 後の観測                        | wrapper の設定が効いているか         |
| estimate_*     | controller 入力                           | controller が何を信じて動いたか      |
| neural_command | 抽象的 controller output                   | 将来の cortical module との接続   |
| excitation     | canonical muscle input                  | SDN 後に何が plant に入ったか       |

|            |                       |                     |
|------------|-----------------------|---------------------|
| api_action | Gym API input         | ActionAdapter の変換確認 |
| activation | muscle internal state | 筋 dynamics の遅れや飽和   |

### 演習 / 作業

closed-loop の結果を読むときは、まず `final_tip_error` を見るのではなく、同じ episode の `true_tip_pos`, `obs_tip_pos`, `estimated_tip_pos`, `excitation` を並べる。どの層で差が生まれたかを見ないと、estimator の効果と controller の癖を混同する。

## 2. 必要な前提知識

### 2.1. 筋骨格 reaching と MyoSuite

MyoSuite myoArm は、MuJoCo 上で動く筋骨格腕モデルである。Project 1 で主に扱うのは、示指先端を target に近づける reaching task である。

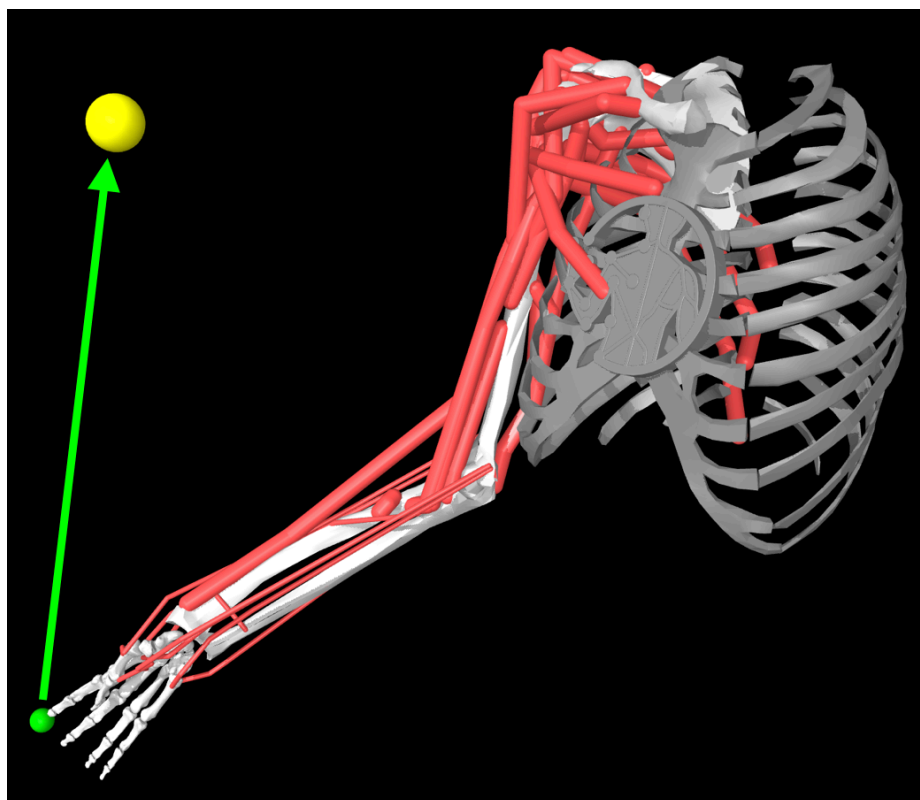


図 2: myoArm reaching task の概念図。リポジトリ内 primer の既存画像を使用。

### 落とし穴

この図の胸部付近にある円形マークは、元画像に含まれるロゴ / 透かしであり、筋・骨・関節・センサーなどのモデル要素ではない。著作権表示や出典表示を意図したマークを削除・改変して使ってはいけない。投稿・配布用の図では、(1) 元画像を改変せず出典とライセンスに従って使う、または (2) 自分で MuJoCo からレンダリングした図に差し替える、のどちらかにする。

| 項目             | 値 / 形                                   | 注意                                      |
|----------------|-----------------------------------------|-----------------------------------------|
| muscles        | 34                                      | action / excitation dimension           |
| joint position | $\mathbf{q}$ in $\mathbb{R}^{20}$       | MyoSuite / MuJoCo の <code>qpos</code>   |
| joint velocity | $\dot{\mathbf{q}}$ in $\mathbb{R}^{20}$ | <code>qvel</code>                       |
| activation     | $\mathbf{a}$ in $\mathbb{R}^{34}$       | 筋内部状態。command ではない                      |
| control step   | 0.02 s                                  | MuJoCo internal timestep 0.002 s と混同しない |



|         |                  |                          |
|---------|------------------|--------------------------|
| episode | 600 steps = 12 s | horizon と max_steps を揃える |
|---------|------------------|--------------------------|

### 落とし穴

MyoSuite の observation は便利だが、研究用の状態 schema と一致するとは限らない。本研究では mj\_data から true state を抽出し、MyoArmState として明示的に管理する。

## 2.2. 信号名の区別

この研究で最も重要な実装原則は、作用信号の層を混同しないことである。

|                                                                                               |                                                                            |
|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>neural_command</b><br>controller が出す抽象的な運動指令。将来、cortical population dynamics の出力になる可能性がある。 | <b>excitation</b><br>研究側 canonical 表現。筋 actuator 入力として解釈する $[0,1]^{34}$ 。  |
| <b>api_action</b><br>Gym / MyoSuite の env.step() に渡す $[-1,1]^{34}$ 。                          | <b>activation</b><br>筋モデル内部の活性化状態。controller output ではない。                  |
| <b>mj_data.ctrl</b><br>MuJoCo の actuator control。post-step inspection 用。                      | <b>true_state</b><br>シミュレータ内部の真値。oracle baseline と評価以外では controller に渡さない。 |

基本変換は次である。

```
api_action = 2.0 * np.clip(excitation, 0.0, 1.0) - 1.0
excitation = (np.clip(api_action, -1.0, 1.0) + 1.0) / 2.0
```

### 確認ポイント

論文・コード・ログで action とだけ書かれていたら、そのたびに「どの層の action か」を確認する。特に SDN は api\_action ではなく neural\_command / excitation 側に入れる。

## 2.3. 状態ベクトル

Project 1 の状態は次の 83 次元ベクトルである。

```
x_t = [qpos, qvel, act, tip_pos, target_pos, reach_err]
dim = 20 + 20 + 34 + 3 + 3 + 3 = 83
```

ここで reach\_err は、現在の論文では target\_pos - tip\_pos として扱う。

| field      | dim | 意味                   |
|------------|-----|----------------------|
| qpos       | 20  | 関節位置                 |
| qvel       | 20  | 関節速度                 |
| act        | 34  | 筋 activation         |
| tip_pos    | 3   | 示指先端位置               |
| target_pos | 3   | target 位置            |
| reach_err  | 3   | target_pos - tip_pos |

## 3. Forward model

### 3.1. なぜ forward model が必要か

感覚 feedback に delay がある場合、時刻  $t$  に届く observation は過去の状態  $x_{(t-d)}$  に対応する。したがって controller が現在状態  $x_t$  に基づいて動くには、過去の観測を forward model で現在まで転がす必要がある。

Oracle Kalman gain across the stress grid by forward-model supervision

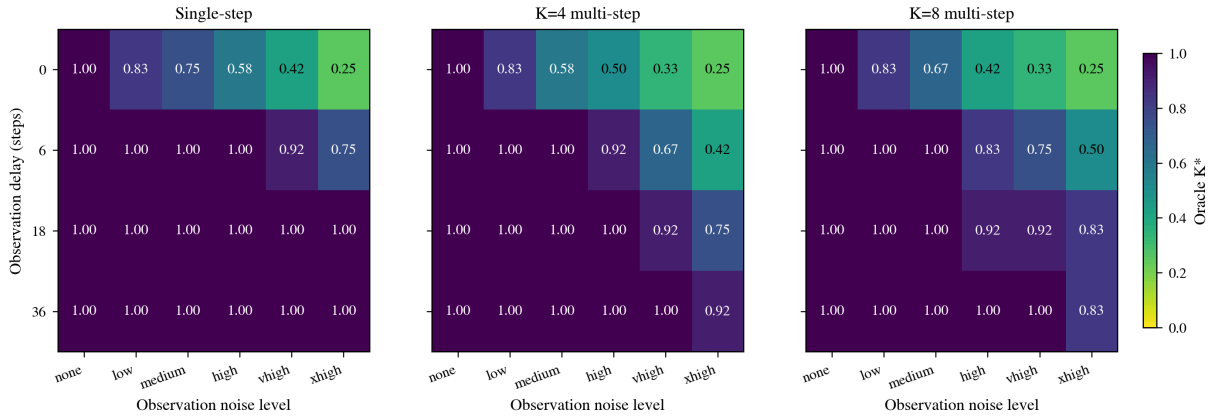


図 3: open-loop oracle gain の heatmap。forward model の訓練方法により、どの delay/noise 条件で prediction を信じるべきかが変化する。

### 3.2. 残差 MLP

Project 1 の baseline forward model は residual MLP である。

input:  $[x_t, u_t]$  in  $\mathbb{R}^{(83 + 34)}$

output:  $\delta x_t$  in  $\mathbb{R}^{83}$

predict:  $\hat{x}_{t+1} = x_t + f_{\theta}(x_t, u_t)$

残差形式を使う理由は、control step が短く、隣接状態の差分が小さいためである。モデルは identity map を再学習するのではなく、状態変化だけを学ぶ。

### 3.3. one-step と multi-step supervision

one-step loss:

$$L_1(\theta) = \mathbb{E} \left[ \left\| f_{\theta}(x_t, u_t) - (x_{t+1} - x_t) \right\|^2 \right]$$

multi-step rollout loss:

$$L_H(\theta) = \mathbb{E} \left[ \frac{1}{H} \sum_{h=1}^H \left\| \hat{x}_{t+h} - x_{t+h} \right\|^2 \right]$$

ここで  $H$  は rollout supervision horizon である。論文では、correction gain の  $K$  と混同しないよう、rollout horizon を  $H$  と呼ぶ。

| 訓練    | 利点                                       | リスク                                              |
|-------|------------------------------------------|--------------------------------------------------|
| $H=1$ | 実装が簡単、one-step MSE が下がりやすい               | long rollout で誤差が蓄積                              |
| $H=4$ | 短期 rollout を安定化                          | 訓練が重い                                            |
| $H=8$ | closed-loop で prediction-only が使える領域が広がる | open-loop oracle と closed-loop optimum の乖離が大きくなる |

#### 研究ノート

Project 1 の重要な発見は、forward model を強くすると「よりよい blending」が得られるだけでなく、closed-loop objective では  $K=0$  prediction-only が支配的になる条件が現れることだった。

## 4. Predictive state observer

### 4.1. 基本式

Project 1 の推定器は、完全な Kalman filter ではない。神経科学的に重要なのは Kalman 最適性ではなく、efference copy に基づく forward prediction と、遅延・ノイズ付き sensory feedback から生じる sensory prediction error を、closed-loop 行動に有用な形で統合できるかである。

最小形の update は次である。

$$\hat{x} = x_{\text{pred}} + K(y - x_{\text{pred}})$$

ここでは  $K$  は  $[0,1]$  の scalar として全 field に broadcast する。 $K$  は Kalman covariance から計算した gain ではなく、sensory prediction error correction gain である。

#### 落とし穴

この推定器を「Kalman filter」と呼ばない。共分散  $P$ 、process noise  $Q$ 、observation noise covariance  $R$  を伝播して最適 gain を計算しているわけではない。本文では predictive state observer または forward-model-based observer と呼び、Kalman は「innovation update と似た形を持つ」という補足に留める。

この式の各項は次のように読む。

| 項                     | 意味                                | 注意                           |
|-----------------------|-----------------------------------|------------------------------|
| $x_{\text{pred}}$     | forward model だけで作った予測            | まだ観測で補正していない                 |
| $y$                   | 遅延・ノイズ付き観測                        | delay がある場合は現在ではなく過去の状態を見ている |
| $y - x_{\text{pred}}$ | sensory prediction error、予測と観測のずれ | 神経科学的には感覚誤差補正信号として読む         |
| $K$                   | prediction error をどれだけ推定に反映するか    | 0 なら感覚誤差を無視、1 なら観測に寄せる       |
| $\text{hat}(x)$       | 補正後の推定状態                          | controller に渡す状態             |

したがって  $x_{\text{pred}}$  は estimator の途中結果であり、 $\text{hat}(x) / \text{xhat}$  は estimator の出力である。

| $K$         | 意味               | 直感                                |
|-------------|------------------|-----------------------------------|
| 0           | prediction-only  | forward model を完全に信じる             |
| 1           | observation-only | delayed/noisy observation を完全に信じる |
| $0 < K < 1$ | blending         | prediction と observation を混ぜる     |

## 4.2. fixed-lag delay handling

delay が  $d$  step ある場合、観測  $y_t$  は現在ではなく  $t-d$  の状態に対応する。そこで estimator は過去の estimate と action buffer を保持する。

buffer:  $\text{xhat}_{(t-d)}, \dots, \text{xhat}_{(t-1)}$

obs:  $y_t \sim x_{(t-d)} + \text{noise}$

1. delayed buffer entry  $\text{xhat}_{(t-d)}$  を observation で correction
2. corrected  $\text{xhat}_{(t-d)}$  を action buffer で  $t$  まで re-roll
3. controller は present estimate  $\text{xhat}_t$  を使う

#### 落とし穴

delay wrapper の意味を変えない。Project 1 では  $\text{observe}(s_t) \rightarrow s_{(t-d)}$  とし、delay buffer length と correction semantics を一貫させる。

## 4.3. 二層 reliability-adaptive observer

ここまでの定式化では  $K$  は固定 scalar である。Project 1 の中心提案は、 $K$  を agent 自身が online で生成できる信号から動的に決める二層適応則である。

#### 二層適応則の骨格

1. Within-trial layer: 各 sensory field の innovation 二乗を EMA で追跡  $\rightarrow$  reliability  $r_{f(t)} \rightarrow$  logistic で per-field gain  $K_{f(t)}$ 。
2. Across-trial layer: SPSA で meta-parameter  $\beta = \{\beta_{0,f}, \beta_{1,f}\}$  を per-episode reaching outcome から更新。

#### agent-available signals

agent 自身が観測・計算可能なシグナルのこと。y\_t(観測)、xhat\_(t-d)(自分の推定)、u\_t(自分の出した運動指令)、reach 終了後の min\_t |tip - target| などが含まれる。x\_t(真の状態)や per-condition の K-sweep oracle ラベルは含まれない。

#### 4.3.1. Within-trial layer: innovation history から reliability へ

各 step、観測 y\_t と過去 estimate の差分(innovation)を計算する。

$$e_t; =; y_t - \hat{x}_{t-d}$$

これは古典的 Kalman の innovation と同じ residual である。Project 1 ではこの 83 次元 vector を 5 つの sensory field に分けて扱う。

| field     | 次元 | 生物学的読み(coarse)                  |
|-----------|----|---------------------------------|
| qpos      | 20 | proprioceptive(joint position)  |
| qvel      | 20 | proprioceptive(joint velocity)  |
| act       | 34 | efferent / muscle-command-like  |
| tip_pos   | 3  | visual / task-space             |
| reach_err | 3  | visual / task-space(target との差) |

target\_pos(3 次元)は信頼度学習の対象外。target は既知量で prediction error が意味を持たない。

各 field の 2 乗 innovation 平均を EMA で追跡する。

$$v_{f(t)}; =; (1 - \alpha), v_{f(t-1)}; +; \alpha c \cdot \text{mean}_{i \in \mathcal{I}_f}(e_{t,i}^2)$$

alpha=0.05 だと時定数 20 step、1 episode の 3% 程度の窓。v\_f が小さい = innovation が一貫して小さい = sensor 信頼できる。逆に大きい = sensor 信頼できない。

reliability に変換:

$$r_{f(t)}; =; \frac{1}{\varepsilon + v_{f(t)}}$$

そして logistic で per-field gain  $K_{f(t)}$  に写像:

$$K_{f(t)}; =; \sigma(\beta_{0,f} + \beta_{1,f}, \log r_{f(t)})$$

| parameter     | 意味        | 直感                                                                                                                            |
|---------------|-----------|-------------------------------------------------------------------------------------------------------------------------------|
| $\beta_{0,f}$ | intercept | $r_f = 1(\log r = 0)$ 時の baseline gain $K_f = \sigma(\beta_{0,f})$ 。例えば $\beta_{0,f} = -1.5$ なら baseline $K_f \approx 0.18$ 。 |
| $\beta_{1,f}$ | slope     | reliability の変化に gain がどれだけ強く反応するか。 $\beta_{1,f} = 0$ なら gain は固定、 $\beta_{1,f}$ が大きいほど innovation 増減で gain が大きく動く。           |

#### 直感

innovation が暴れる → reliability が下がる →  $K_f$  が小さくなる → sensor を信じず forward model を信じる。

innovation が落ち着く → reliability が上がる →  $K_f$  が大きくなる → sensor を信じる。

この per-field 動的調節を innovation history だけ から行うのが within-trial layer。

#### 4.3.2. Across-trial layer: SPSA outcome adaptation

within-trial layer の挙動は  $\beta = \{\beta_{0,f}, \beta_{1,f}\} \in \mathbb{R}^{10}$  に支配される。  $\beta$  をどう決めるかは、reach 終了後の reaching outcome から学習する。

outcome は per-episode の minimum tip-to-target distance:

$$\text{minTip}(\beta); =; \min_{t \in [0, T]} |p_{\text{tip}}(t) - p_{\text{tgt}}|$$

これは生体の場合「reach 後どこに着いたかの視覚 + proprioception 由来の評価」に対応する scalar。minTip 自体は biological claim ではなく、agent-available な outcome の operational proxy として使う。

closed-loop minTip は  $\beta$  に対して解析的勾配を取れないので(env step  $\rightarrow$  EMA  $\rightarrow$  sigmoid  $\rightarrow$  control  $\rightarrow$  non-smooth min を経由する)、SPSA(Spall 1992)で勾配 free 最適化する。

SPSA 1 iteration:

```
Delta_n ~ uniform([-1, +1])^10 // Rademacher perturbation
o_plus = minTip(beta_n + c_n * Delta_n) // S 個 episode 平均
o_minus = minTip(beta_n - c_n * Delta_n) // S 個 episode 平均
g_hat = (o_plus - o_minus) / (2 c_n) * Delta_n^{-1}
beta_{n+1} = beta_n - a_n * g_hat
```

Spall schedule:

```
a_n = a / (n + 1 + A)^alpha_s
c_n = c / (n + 1)^gamma_s
```

defaults: a=2.0, c=0.3, alpha\_s=0.602, gamma\_s=0.101, A=5, S=10 or 12。

#### Black-box stochastic approximation

最適化対象の中身を見ずに、入力を振動して出力(scalar evaluation)を観測するだけで勾配推定する手法群。SPSA はそのうち最も次元スケーラブルなアルゴリズム。policy search / Evolution Strategies と同じカテゴリ。

Project 1 では SPSA を 2 つの設定で走らせた:

| variant     | training cells                                               | 使い道                                                               |
|-------------|--------------------------------------------------------------|-------------------------------------------------------------------|
| single-cell | ( $\sigma = \text{none}$ , $d = 18$ ) の 1 cell のみ            | agent-available adaptation が closed-loop oracle に追いつけるかの存在証明 (C2) |
| full-grid   | 3 noise $\times$ 2 delay = 6 cell から uniform random sampling | single global $\beta$ で grid を覆えるかの structural test (C3)          |

#### 4.3.3. oracle-supervised predictor: upper bound diagnostic (Appendix C 教材)

提案手法と対比される旧 main contribution(現 Appendix C)は、condition feature (sigma, d, c) から scalar K を出す MLP を、per-condition の K-sweep oracle ラベルで supervised に学習する。

$g\_phi : (\text{one\_hot}(c), \text{sigma}, d/d\_max) \rightarrow K \text{ in } [0, 1]$

oracle ラベルは 2 種類:

| oracle              | 定義                                              | 最適化するもの                          |
|---------------------|-------------------------------------------------|----------------------------------|
| $K^*_{\text{"ol"}}$ | $\text{argmin}_K E[\ \hat{x}_{t(K)} - x_t\ ^2]$ | open-loop state estimation error |
| $K^*_{\text{"cl"}}$ | $\text{argmin}_K E[\text{minTip}(K)]$           | closed-loop reaching error       |

#### Appendix C 教材としての位置

oracle-supervised predictor は agent-available ではない: per-condition の K-sweep を回す段階で生体には不可能な data 収集を要求する。提案手法と直接比較するための non-agent-available upper bound として Appendix C に残してある。

重要な観察: closed-loop oracle  $K^*_{\text{cl}}$  と open-loop oracle  $K^*_{\text{ol}}$  は乖離することがあり、特に multi-step supervision された forward model 下で顕著(closed-loop で  $K = 0$  が optimal なのに、open-loop oracle では  $K = 1$  が optimal)。これは learned correction gain を評価する際に 何を教師 oracle としたかを明示すべきだという、Appendix C の中心的指摘である。

## 5. 実験 pipeline

### 5.1. Repository の構造

src/myoarm\_fse/      Python package  
scripts/            CLI scripts  
configs/            YAML configs  
runs/               local outputs; large artifacts are not versioned  
tests/              unit and smoke tests  
docs/               plans, primers, this textbook  
figures/            paper figures and CSV summaries  
paper/              TNNLS manuscript

環境構築とテストは uv を使う。

```
uv sync
uv run pytest
uv run pytest -m myosuite
```

#### 落とし穴

Python を使う場合は python 直実行ではなく uv run python ... を使う。依存関係と仮想環境を固定するためである。

### 5.2. 最小再現手順

研究を最初から再現する場合の概念的な手順は以下である。実際のファイル名は configs/ と scripts/ を確認する。

| Step | 作業                            | 確認する出力                                   |
|------|-------------------------------|------------------------------------------|
| 1    | target set 生成                 | runs/targets/*.npz                       |
| 2    | episode collection            | runs/episodes/<timestamp>/               |
| 3    | TransitionDataset 作成 / concat | runs/datasets/*.npz                      |
| 4    | forward model training        | runs/models/<timestamp>/                 |
| 5    | open-loop estimator sweep     | runs/estimators/<timestamp>/metrics.csv  |
| 6    | learned gain training         | runs/learned_gain_models/<timestamp>/    |
| 7    | closed-loop evaluation        | runs/closed_loop/<timestamp>/metrics.csv |
| 8    | paper figures                 | figures/F*.png, figures/data/*.csv       |

### 5.3. 実験 command の読み方

代表的な command は次の形になる。

```
uv run python scripts/train_forward_model.py \
  --config configs/models/mlp_expanded.yaml

uv run python scripts/evaluate_estimator.py \
  --config configs/estimators/fixed_kalman_robustness.yaml \
  --forward-model runs/models/<model-id>

uv run python scripts/evaluate_closed_loop.py \
  --config configs/closed_loop/<config>.yaml
```

設定 YAML を変えるときは、実験条件を run directory に保存し、後で paper figure を再生成できるようにする。

## 6. 主要結果の読み方

ここまでで、closed-loop pipeline、forward model、predictive state observer、二層適応則(within-trial reliability + across-trial SPSA)、そして Appendix C の oracle-supervised upper bound を説明した。ここからは、その道具を使って論文の主張 C1-C4 を読む。



| Claim | 内容                                                                                                                                                                 | 意味                                                                     |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| C1    | default within-trial reliability ( $\beta_{0,f} = 0, \beta_{1,f} = 0.5$ )は field-wise heterogeneous な中-高 gain を出し、 $H = 8$ 下では task-optimal $K = 0$ から 23-29 cm 外す | innovation だけでは task を知らない                                             |
| C2    | single-cell SPSA は trained cell 上で default-reliability vs $K = 0$ gap の 23 cm のうち $\approx 17$ cm を回復( $0.77 \rightarrow 0.56$ m vs $K = 0$ at 0.50 m)             | outcome layer を足せば agent-available signal だけで closed-loop optimum に近づく |
| C3    | multi-cell SPSA は delay-18 cells で 3-5 cm の改善どまり、delay-0 cells で改善ゼロ。SPSA の失敗ではなく、global single- $\beta$ の parameterisation が underpowered                         | 次の model class は context-conditioned $\beta$                           |
| C4    | ReachFixed-trained $\beta$ を ReachRandom に転移すると全 delay-18 cells で $K = 0$ を $\sim 4-5$ cm 下回る。default reliability の方が逆に競合的                                         | correction-gain rule は task-dependent ( $K_{cl}^*$ geometry に依存)       |

### 6.1. C1: default reliability is task-misaligned

F\_reliability\_default は、 $H = 8$  forward model + joint-PD controller 下で  $K = 0$  /  $K = 1$  / default reliability( $\beta_{0,0}=0, \beta_{1,0}=0.5$ )の closed-loop min-tip を 6 cell で比較した結果である。

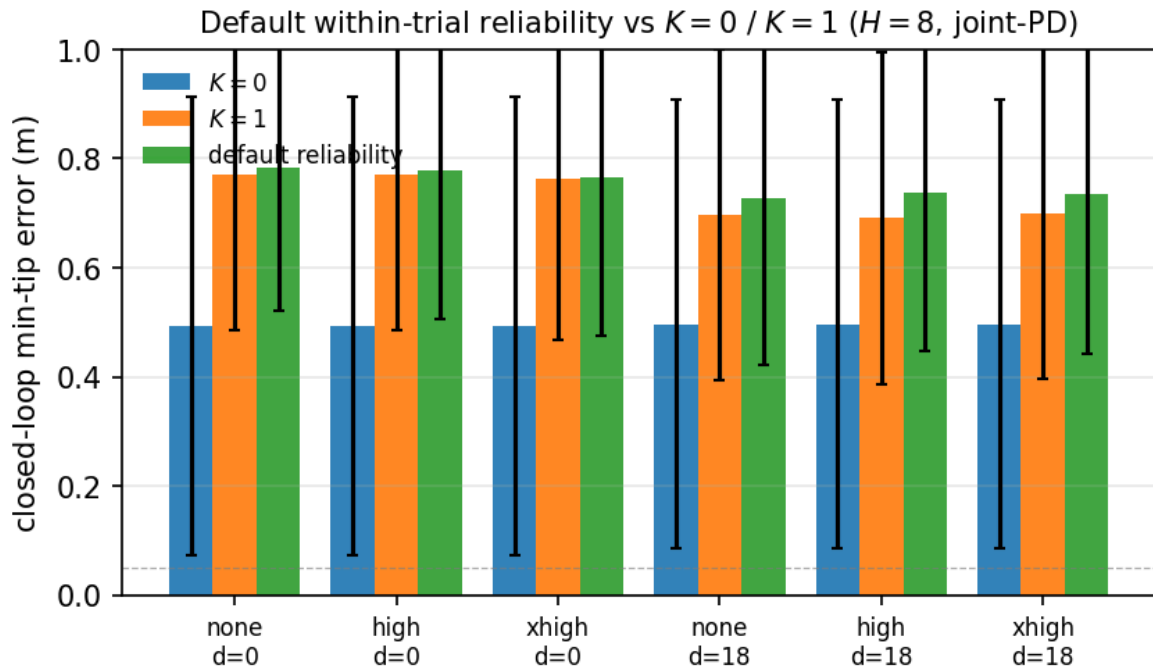


図 4: default within-trial reliability は  $K=0$  を 23-29 cm 下回る。  $K_{cl}^*=0$  一様な ReachFixed grid 下で innovation reliability だけでは task-optimal な gain を出せない構造的限界を示す。

実 diagnostic dump から、default rule が出す per-field  $K_f$  は強く非対称:

| cell        | qpos | qvel | act  | tip_pos | reach_err |
|-------------|------|------|------|---------|-----------|
| none, d=0   | 0.96 | 0.49 | 0.91 | 0.97    | 0.98      |
| none, d=18  | 0.70 | 0.31 | 0.69 | 0.59    | 0.68      |
| high, d=0   | 0.96 | 0.49 | 0.92 | 0.96    | 0.97      |
| high, d=18  | 0.69 | 0.32 | 0.71 | 0.68    | 0.69      |
| xhigh, d=0  | 0.90 | 0.45 | 0.91 | 0.94    | 0.94      |
| xhigh, d=18 | 0.67 | 0.31 | 0.69 | 0.64    | 0.68      |

つまり default rule のもとでは、qvel だけが一貫して低 gain (0.31-0.49)、他 4 field は中-高 gain (0.59-0.98) に張り付く。innovation reliability は task を知らないのも、 $K_{cl}^* = 0$  という task-optimal な水準には届かない。

### C1 の核心

innovation だけ追っても、forward model がどれくらい task に有用かは分からない。  
sensor 信頼度と “forward model に任せて良いか” は別の情報。  
後者は task outcome を見ないと決まらない。

## 6.2. C2: single-cell SPSA closes most of the gap

F\_spsa\_single は、( $\sigma = \text{none}, d = 18$ ) という 1 cell 上で SPSA を 100 iteration 回した結果である。

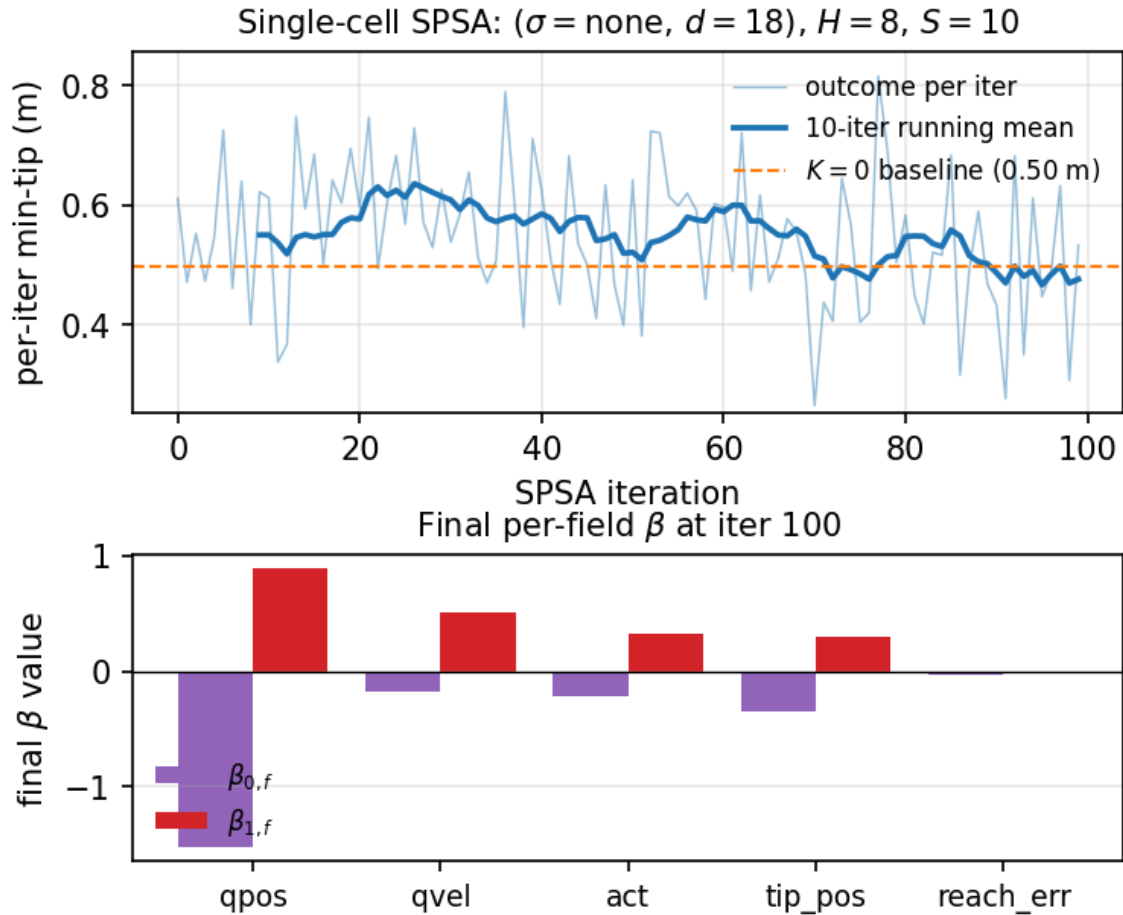


図 5: 上: per-iter outcome trajectory(青)+ 10-iter running mean(濃青)+  $K = 0$  baseline 0.50 m(橙破線)。下: final  $\beta$ 。qpos は intercept ( $-1.5$ )も slope ( $+0.9$ )も先頭で、field-wise specialisation が outcome から自動生成される。

主要数値:

- ・ per-iter outcome は  $\approx 0.61 \text{ m} \rightarrow \approx 0.53 \text{ m}$  に下降(last iter)
- ・ 10-iter running mean(訓練時 smoothed signal)は  $K = 0$  baseline 0.50 m まで一瞬到達
- ・ 別途 fresh deployment(final  $\beta$  を独立 10 episode に適用)で min-tip = 0.56 m  $\rightarrow$  residual  $\sim 6 \text{ cm}$
- ・ default reliability(0.77 m)と比べて  $0.77 \rightarrow 0.56$  の 17 cm gap recovery

### C2 の核心

agent は K-sweep oracle ラベルを使わずに、per-episode reaching outcome だけから  $\beta$  を更新できる。  
trained cell では default reliability gap の  $\approx 74\%$  を SPSA で閉じる。  
field-wise specialisation(qpos が深く suppressed、reach\_err slope が flat 化)は outcome から自然と出る。

## 6.3. C3: multi-cell SPSA reveals parameterisation ceiling

F\_spsa\_fullgrid は、SPSA を 6 cell uniform-random sampling で 100 iteration 回した結果である。

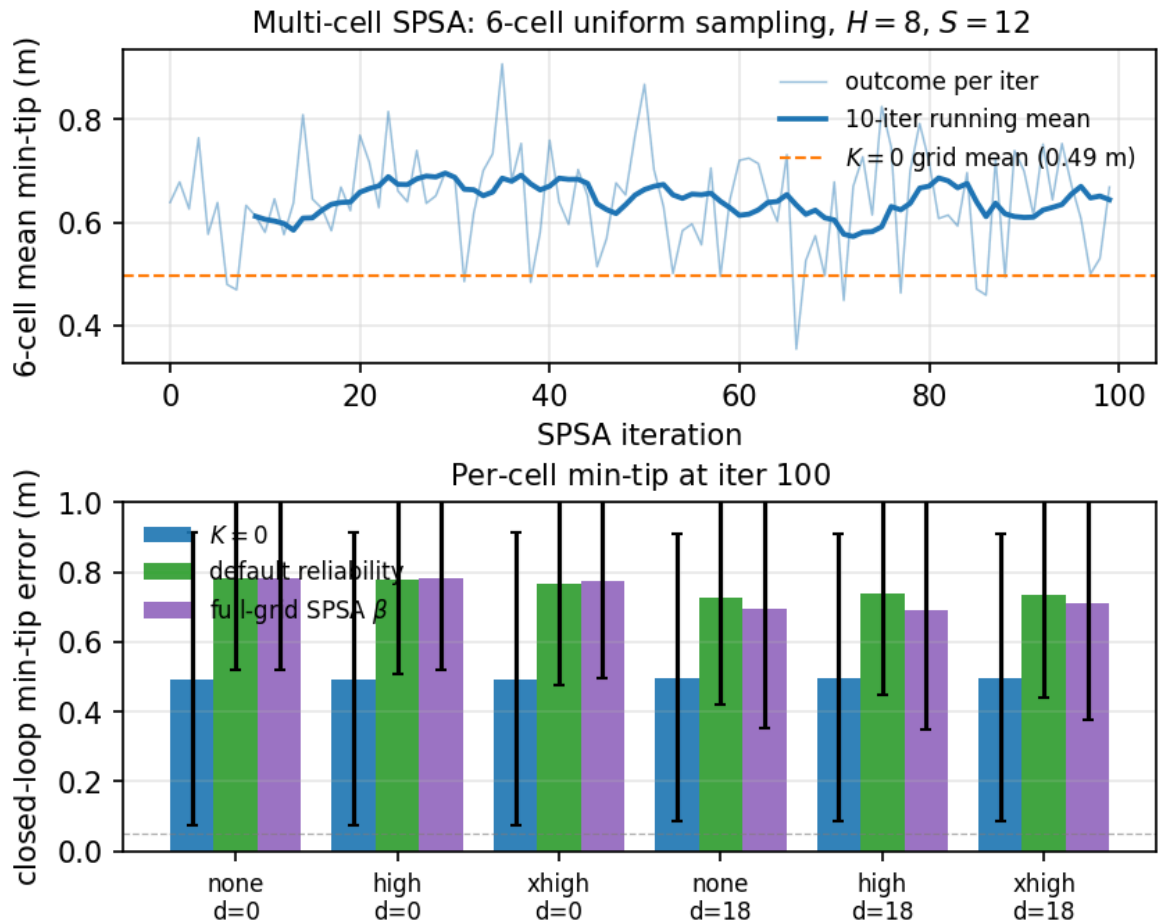


図 6: 上: 6-cell mean outcome trajectory.  $K = 0$  grid mean 0.49 m(橙破線)に届かない。下: per-cell min-tip. delay-18 cells で 3-5 cm の改善、delay-0 cells で改善ゼロ。  $K = 0$  からは 19-22 cm( $d=18$ ) /  $\sim 28$  cm( $d=0$ ) 離れたまま。

主要観察:

- delay-18 cells: 3 cells すべてで default reliability より 3-5 cm 改善。  $K = 0$  から 19-22 cm trail。
- delay-0 cells: SPSA はほとんど改善せず、  $K = 0$  から  $\sim 28$  cm trail。
- 全 cell で  $K_{cl}^* = 0$  なのに、 single global  $\beta$  では reach できない構造的限界。

### C3 の核心: SPSA の失敗ではない

single-cell SPSA(C2)は確かに収束した。 problem は SPSA loop 自体ではなく、  $\beta \in \mathbb{R}^{10}$  という single-global parameterisation が、 cell ごとに異なる innovation geometry を 同時に 抑えられないこと。

高ノイズ cell で  $K_f$  を 0 まで押し下げる  $\beta_{0,f}$  を選ぶと、低ノイズ cell では over-suppress、逆も真。一つの  $\beta$  で全 cell の “trust the model” を出すには、reliability の絶対水準を見ず context を見る必要がある。

natural next model:

```
beta_network(
  sliding window of field-wise innovation statistics
    (mean, variance, autocorrelation)
) -> beta in  $\mathbb{R}^{10}$ 
```

(sigma, d) を auxiliary input にすることもできるが、生体は noise/delay label を直接観測しないので biological framing では innovation window が primary input。

## 6.4. C4: task transfer reveals task-dependent rule

ReachRandom(target が episode 毎にランダム抽出)に同じ observer を転移すると、興味深い反転が起きる。

| cell        | $K = 0$ | $K = 1$ | default | transferred $\beta$ | $K_{cl}^*$ |
|-------------|---------|---------|---------|---------------------|------------|
| none, d=0   | 0.670   | 0.770   | 0.781   | 0.768               | $K = 0$    |
| none, d=18  | 0.642   | 0.647   | 0.631   | 0.680               | $K = 0.25$ |
| high, d=0   | 0.670   | 0.770   | 0.776   | 0.768               | $K = 0$    |
| high, d=18  | 0.642   | 0.639   | 0.608   | 0.692               | $K = 0.25$ |
| xhigh, d=0  | 0.670   | 0.762   | 0.771   | 0.759               | $K = 0$    |
| xhigh, d=18 | 0.642   | 0.627   | 0.649   | 0.686               | $K = 1$    |

数値の読み方:

- ReachRandom の  $K_{cl}^*$  は  $\{0, 0.25, 1\}$  の 3 値を取る multimodal 構造 (Appendix E)。
- delay-18 cells で default reliability が  $K = 0$  を勝つ (2 of 3 cells で 1-3 cm)。default rule の  $K_f \approx 0.6-0.7$  が偶然 per-cell  $K_{cl}^* = 0.25$  に近いため。
- 一方 ReachFixed-trained  $\beta$  は全 delay-18 cell で  $K = 0$  を  $\sim 4-5$  cm 下回る。source task では  $K_{cl}^* = 0$  一様だったので  $\beta$  が gain を global に下げる方向に学習されており、target task の multimodal  $K_{cl}^*$  には合わない。

### C4 の核心: rule は task-dependent

correction-gain rule の “正解” は task 構造(= $K_{cl}^*$  geometry)に依存する。

$K_{cl}^*$  が uniform な task では outcome adaptation が closes the gap。

$K_{cl}^*$  が multimodal な task では default reliability の方が逆に競合的(中-高 gain が偶然 multimodal optima に近い)。

testable computational hypothesis: uniform-optimum task で trained agent は multimodal-optimum variant に転移すると性能を落とすはず、context-conditioned rule でない限り。

## 7. 研究者としての作業手順

### 7.1. 1 日の作業サイクル

1. 研究問いを 1 文で書く
2. config を 1 つだけ変える
3. 実行前に expected outcome と fail condition を書く
4. uv run で実行
5. metrics.csv / summary.json を確認
6. 図表を再生成
7. Logs または exchange に判断を書く
8. commit する

### 7.2. 実験前チェックリスト

#### 演習 / 作業

- ☐ env id は意図通りか (myoArmReachFixed-v0 / myoArmReachRandom-v0)
- ☐ target は paired か、direct-write されているか
- ☐ downstream policy は true state を見ていないか
- ☐ policy 名は joint-space feedback controller / endpoint-error sensorimotor policy / behaviour-cloned policy のどれかに整理されているか
- ☐ action layer (excitation, api\_action, activation) が混ざっていないか
- ☐ delay semantics は過去ノートと一致しているか
- ☐ run directory に resolved config が保存されるか
- ☐ random seed / target index / model id が metrics に残るか

## 7.3. 結果解釈チェックリスト

### 演習 / 作業

- [ ] state estimation error と task error を混同していないか
- [ ] open-loop oracle と closed-loop oracle を分けているか
- [ ] paired comparison になっているか
- [ ] single-cell の大きな差を一般化しすぎていないか
- [ ] downstream policy の state coupling を評価しているか
- [ ] observer sensitivity を reaching performance と混同していないか
- [ ] negative result を捨てずに設計上の知見として記録しているか

## 8. よくある失敗と対処

### 8.1. target が paired でない

myoArmReachRandom-v0 では、単に `env.reset(seed=k)` するだけでは target 再現性が保証されない場合がある。Project 1 では target set を自前で持ち、target site を direct-write する。

```
env.reset()
_inject_target(env, target_set.target_pos[episode_index])
mujoco.mj_forward(model, data)
```

paired target が崩れると、estimator 差ではなく target distribution 差を測ってしまう。

### 8.2. K と H の混同

correction gain の K と rollout supervision horizon の H を混同しない。

K: sensory prediction-error correction gain,  $0 \leq K \leq 1$

H: rollout supervision horizon, H in {1, 4, 8}

d: observation delay steps

### 8.3. behaviour-cloned policy の open-loop 化(Appendix C 教材)

BC は reaching を改善するが、小規模 demo では平均 trajectory を再生するだけになりやすい。この場合、observer output を入力に入れていても、実質的には state-coupled ではない。Project 1 の main result では joint-PD controller を採用し、BC は Appendix C の oracle-supervised diagnostic に降格してある(observer signal の wash-out を示す methodological caution として残す)。

対処:

- DAgger / closed-loop data aggregation
- target-conditioned policy の明示
- state masking の効果検証
- observer sensitivity metric の導入

### 8.4. endpoint-error policy を optimal controller と呼ぶ

endpoint-error sensorimotor policy は、estimated tip-to-target error を muscle excitation に写像する task-level policy である。LQR、OFC、Riccati equation は解いていない。Project 1 main では joint-PD のみを採用し、endpoint-error policy は Appendix C(oracle-supervised diagnostic)で「joint-PD 以外の state-coupled policy でも同じ qualitative finding が出ること」の確認用として残してある。

対処:

- endpoint-error sensorimotor policy または endpoint-error policy と呼ぶ
- OFC-like, LQR-like, optimal feedback controller と書かない
- absolute reaching と state coupling を分けて評価する
- $K=0$  /  $K=1$  / learned で trajectory と action が分かれるかを見る

## 8.5. SPSA outcome を training-time と deployed eval で混同する

SPSA の per-iter outcome は同じ  $\beta$  で  $s$  個 episode 平均しているが、これは training-time の noisy estimate である。fresh deployment では別 seed の独立 episode で測り直す必要がある。

training-time outcome (smoothed last-20-iter mean) vs  
deployed eval (final beta, fresh 10 episodes)

Project 1 の C2 では両者を分けて報告している (0.49 m smoothed training vs 0.56 m deployed)。performance claim は deployed eval が headline、smoothed training は convergence evidence。

## 8.6. oracle なしを “no oracle access” と書く

abstract / intro で “no oracle access” / “without an offline oracle” と書くと、reviewer に “実用システムは当然 oracle 使えないので trivial” と読まれる。実際の non-trivial claim は training 時に per-condition K-sweep oracle ラベルを要求しないこと。

対処:

- “without per-condition oracle labels at training time”
- “rather than oracle-supervised gain learning”
- Appendix C への明示ポイントを併記

## 9. Project 2 / Project 3 への接続

### 9.1. Project 1.5: context-conditioned $\beta$ network

C3 の structural ceiling は、natural な follow-up project として「context-conditioned  $\beta$  学習器」を要請する。

beta\_network:

input = sliding window of field-wise innovation statistics  
(mean, variance, autocorrelation; optionally (sigma, d)  
as ablation-only auxiliary)  
output = beta = {beta\_0\_f, beta\_1\_f} in  $\mathbb{R}^{10}$

学習 signal は引き続き per-episode reaching outcome を SPSA / REINFORCE / ES のような black-box optimization で用いる (target K のラベルは依然として使わない)。

設計上の question:

- innovation window 長: episode 全体 vs 直近 50-100 step
- context encoder: small MLP / GRU / Transformer
- network output:  $\beta$  そのもの /  $\beta$  への residual / per-step  $K_f$  直接
- ReachFixed train  $\rightarrow$  ReachRandom test の transfer benchmark を pre-commit

これは Project 1 の論文 main scope の外だが、C3 の限界を直接解消する次の研究軸として明示。

### 9.2. Project 2: Bayesian integration

Project 1 では observation noise を field ごとの Gaussian sigma として扱い、within-trial reliability は innovation 二乗から経験的に推定した。Project 2 では、視覚・固有受容・prediction・prior を別情報源として扱う Bayesian framework に拡張する。

Project 1 との橋渡し:

- Project 1 の reliability  $r_{f(t)} = \frac{1}{\varepsilon + v_{f(t)}}$  は noise covariance estimate  $\hat{\sigma}_f^2(t) = v_{f(t)}$  の inverse として読める
- Bayesian integration では  $w_i \propto \frac{1}{\sigma_i^2}$  で weighting  $\rightarrow$  Project 1 logistic を  $\sigma(\beta_0 + \beta_1 \log r)$  と書いた  $\beta_1 = 1, \beta_0 = 0$  の special case として読める
- Project 2 は modality conflict (visual vs proprioceptive)、prior の bias、posterior の uncertainty を陽に扱う

候補実験:

- visual noise と proprioceptive noise を別々に増やす (現 paper は同一  $\sigma$  vector で field-wise scale)

- ・ sensory conflict(visual と proprioceptive を意図的に矛盾させる)
- ・ target prior を偏らせる(ReachRandom の target 分布操作)
- ・ endpoint bias / variability を測る
- ・ 各 source の online estimated precision を beta\_network の input に追加

### 9.3. Project 3: cortico-cerebellar loop

Project 1 の residual MLP は Wolpert/Kawato box-level の forward model 近似としては妥当だが、小脳 microcircuit そのものではない。Project 3 では、forward model を以下のような module に分ける。

```
cortical command
-> pontine-like encoder
-> cerebellar-like forward model
-> thalamic-like relay
-> cortical estimator/controller
```

LTC / CfC は、連続時間・recurrent・入力依存時定数という点で、Project 3 の候補 forward model として自然である。Project 1 の two-layer adaptation(within-trial reliability + across-trial outcome)は、Project 3 の microcircuit module 間でも同じ構造的役割を果たすと想定: cerebellar forward model のシナプス可塑性 = within-trial dynamics、prefrontal / striatal level の outcome-based meta-learning = across-trial。

#### 落とし穴

TNNLS 投稿前に CfC/LTC PoC を始めると、Project 1 の論文主張がぶれやすい。まず Project 1 の投稿を完了し、その後に別ブランチ / 別 phase として Project 1.5(context-conditioned  $\beta$ ) → Project 2(Bayesian) → Project 3(cortico-cerebellar)の順で進める。

## 10. 演習

### 10.1. 演習 1: 状態 schema を確認する

```
uv run python -c "from myoarm_fse.envs.factory import make_env; from myoarm_fse.envs.extractors import extract_state;
env=make_env('myoArmReachFixed-v0'); env.reset(); s=extract_state(env); print(s.flatten().shape)"
```

期待:

(83,)

### 10.2. 演習 2: test を通す

```
uv run pytest
uv run pytest -m myosuite
```

default tests と MyoSuite smoke tests の両方を通す。

### 10.3. 演習 3: 図表を読み解く

F\_reliability\_default / F\_spsa\_single / F\_spsa\_fullgrid を開き、以下を説明せよ。

- ・ F\_reliability\_default で default reliability が  $K=0$  を 23-29 cm 下回る構造的理由は何か(forward model 強度と reliability の関係)
- ・ F\_spsa\_single の running mean が  $K=0$  baseline 0.50 m に到達するのに、deployed eval は 0.56 m に留まる理由は何か(SPSA noise vs eval seed の関係)
- ・ F\_spsa\_single の bottom panel で qpos の  $\beta_0$  が一番低く、 $\beta_1$  が一番高い意味を解釈せよ(field-wise specialisation)
- ・ F\_spsa\_fullgrid で multi-cell SPSA が  $K=0$  に届かない理由を構造的に述べよ(single global  $\beta$  の表現限界)
- ・ Appendix C(oracle-supervised)の  $K_{ol}^*$  と  $K_{cl}^*$  が乖離する条件は何か(forward model multi-step supervision との関係)



## 10.4. 演習 4: 新しい transfer test を設計する

ReachRandom transfer 以外の transfer 軸(controller 変更、forward model H 変更、noise level 拡張、新しい task variant など)を行う場合、実行前に次を文章で固定せよ。

- ・ 何を replication target とするか
- ・ 何を replication target から外すか
- ・ pass / partial / fail の基準
- ・ target pairing の保証方法
- ・  $K_{cl}^*$  structure(uniform / multimodal)の事前予測
- ・ default reliability vs SPSA-trained  $\beta$  vs  $K = 0$  の比較順序
- ・ Appendix に入れる最小図表

## 10.5. 演習 5: context-conditioned $\beta$ の prototype を設計する

C3 ceiling を超える next-step model を設計せよ。最低限決めるべきこと:

- ・  $\beta$  network の入力(sliding window 長、何の statistics か)
- ・  $\beta$  network の出力構造( $\beta$  直接 / residual / per-step  $K_f$ )
- ・ 学習 signal(per-episode outcome、step-wise reward など)
- ・ 最適化手法(SPSA / REINFORCE / ES / 微分可能 surrogate)
- ・ ReachFixed  $\rightarrow$  ReachRandom transfer の評価設定
- ・ failure mode(over-fit、変動 high など)の事前予想

実装まで進めなくてよい。設計を 1-2 ページにまとめることが演習。

# 11. 参考資料

## 11.1. Repository 内

- ・ README.md
- ・ docs/02\_InitialImplementationPlan.md
- ・ docs/03\_PaperOutline.md
- ・ docs/03\_Project1\_Foundations.typ
- ・ paper/main.tex
- ・ figures/F1\_system\_overview.{pdf,png}
- ・ figures/F\_reliability\_default.{pdf,png}
- ・ figures/F\_spsa\_single.{pdf,png}
- ・ figures/F\_spsa\_fullgrid.{pdf,png}
- ・ figures/F2-F7.{pdf,png}(Appendix C oracle-supervised 教材用)
- ・ scripts/make\_reframe\_figures.py(C1-C3 figure 再生成)
- ・ scripts/diagnose\_reliability\_observer.py(per-cell  $K_f$  dump)
- ・ scripts/train\_reliability\_adaptive\_v2.py(SPSA outer loop)
- ・ src/myoarm\_fse/estimators/reliability\_adaptive.py

## 11.2. Obsidian 内

- ・ 20\_research/myoarm-fse/\_index.md
- ・ 20\_research/myoArm新規研究プロジェクト候補\_2026-05-09/00\_全体研究計画.md
- ・ 20\_research/myoArm新規研究プロジェクト候補\_2026-05-09/01\_Project1\_ForwardStateEstimation研究計画.md
- ・ Learn/NeuroControl/小脳forward-modelの数学的実装-residual-MLPとLTC.md

## 11.3. 文献

- ・ Bernstein, N. A. (1967). The Co-ordination and Regulation of Movements.
- ・ Wolpert, D. M., & Ghahramani, Z. (2000). Computational principles of movement neuroscience.
- ・ Kalman, R. E. (1960). A new approach to linear filtering and prediction problems.

- Mehra, R. K. (1970). On the identification of variances and adaptive Kalman filtering(adaptive Kalman = innovation-based covariance estimation の古典).
- Rauch, H. E., Tung, F., & Striebel, C. T. (1965). Maximum likelihood estimates of linear dynamic systems.
- Spall, J. C. (1992). Multivariate Stochastic Approximation Using a Simultaneous Perturbation Gradient Approximation(SPSA の原典).
- Caggiano et al. (2022). MyoSuite.
- Hasani et al. (2021). Liquid Time-constant Networks.
- Hasani et al. (2022). Closed-form Continuous-time Neural Networks.

## 12. 最後に

この研究を遂行するうえで最も大切なのは、実験を増やすことではなく、次の4つを分け続けることである。

### 4つの分離

1. state estimation error と closed-loop task error を分ける。
2. agent-available signals(innovation history, per-episode outcome)と non-agent-available oracle ラベル(per-condition K-sweep の  $K^*$ )を分ける。
3. within-trial layer(per-step reliability)と across-trial layer(episode outcome ベースの  $\beta$  更新)を分ける。
4. SPSA loop convergence(C2 で示した)と parameterisation 表現能力(C3 が拘束されている)を分ける。

この4つを分けて考えれば、Project 1 の実装結果は単なる controller engineering ではなく、forward prediction、innovation-based online reliability、outcome-based across-trial learning の三層構造を持つ生物学的にもっともらしい motor control framework の研究基盤になる。次の自然な拡張は、C3 の structural ceiling を超える context-conditioned  $\beta$  network(Project 1.5)と、それを Bayesian framework に embed する Project 2 である。